Implementation Document

for

QWIK

Version 1.0

Prepared by

Group : 13 **Group Name: Code Crafters -2**

Katroth Chandrakanth	240531	Katrothc24@iitk.ac.in
Dharavath yuvraj singh	240349	Dharavathr24@iitk.ac.in
Kanakala Jashwanth kumar	240513	kanakala24@iitk.ac.in
Kallu Venkata Prasun	240508	kallu24@iitk.ac.in
Boosaraju Sachin	240282	boosarajus24@iitk.ac.in
Modugu Akhilesh Kumar	240656	akhileshmkm24@iitk.ac.in
Cheekatla Rishita diya	240303	diyarc24@iitk.ac.in
Velpula Jashwanth	241157	jashwanthv24@iitk.ac.in
Sumanth Reddy Kamjula	240510	sumanthkr24@iitk.ac.in
Ramalingam Naidu Allu	240843	ramalingam24@iitk.ac.in

Course: CS253

Mentor TA: *Utkarsh Agarwal*

Date: March 27,2026

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1
1.1 MVC ARCHITECTURE FOR QWIK
1.2 INTEGRATION OF MVC WITH LAYERS
1.3 LAYERED BUSINESS LOGIC
1.4 DEVELOPMENT AND VERSION CONTROL ENVIRONMENT
1.5 AUTHENTICATION AND AUTHORIZATION
2 CODEBASE .....

3 COMPLETENESS 3
3.1 FUNCTIONALITIES THAT HAVE BEEN IMPLEMENTED
3.2 NON-FUNCTIONAL REQUIREMENTS ACHIEVED
3.3 FUTURE DEVELOPMENT PLAN
APPENDIX A - GROUP LOG 4

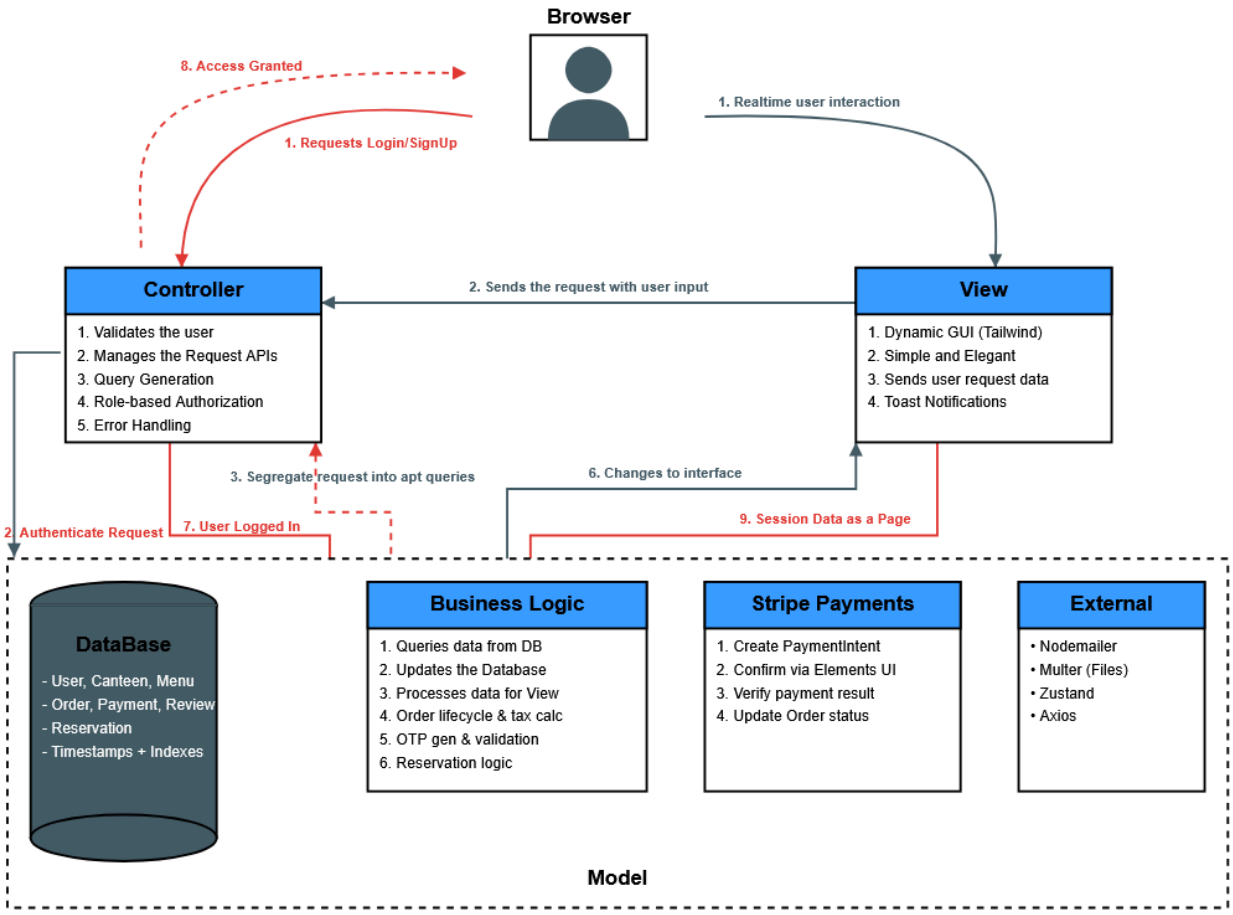
Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft Type and Number	Full Name	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Implementation Details

For the **Qwik** project, we combined the **Model-View-Controller (MVC)** architecture with a **Layered Architecture** to make the system scalable, easy to maintain, and flexible.

- **MVC architecture** divides the application into three main parts:
  - **Model**: Handles data and business logic
  - **View**: Manages the user interface
  - **Controller**: Handles user requests and updates the Model and View
- **Layered Architecture** further organizes the system into:
  - **Presentation Layer**: Responsible for the UI
  - **Business Logic Layer**: Contains core functionality
  - **Data Layer**: Handles database operations



1.1 MVC Architecture for Qwik

1.1.1 Model: Data and Business Logic

The Model represents the core data and business logic of the application. It manages key data such as user information, food orders, and canteen details, and interacts directly with the database.

Responsibilities of the Model:

- User authentication data, OTP lifecycle, and session management.
- Canteen menus, item availability, pricing, and rating aggregation.
- Full order lifecycle including status transitions and partial-accept flows.
- Table and cake-order reservations with advance-booking validation.
- Payment records linked to Stripe Payment Intents.
- Review and rating storage per order item.

We used **MongoDB**, a NoSQL database, along with **Mongoose** for schema modeling and efficient data handling.

Advantages of MongoDB:

- Flexible schema design for dynamic data
- High scalability and performance
- Easy integration with JavaScript-based applications
- Suitable for real-time and large-scale applications

We used **Node.js** as the backend runtime environment.

Advantages of Node.js:

- Fast execution using the V8 engine
- Supports asynchronous and non-blocking operations
- Uses JavaScript across both frontend and backend
- Large ecosystem through NPM
- Highly scalable for real-time applications

1.1.2 View: User Interface and Presentation

The View is responsible for displaying the user interface and presenting data to users. It receives processed data from the Controller and does not directly interact with the Model.

Responsibilities of the View:

- Displaying menus, food items, and canteen details
- Showing user profiles and order history
- Handling user interactions such as login, signup, and order placement
- Dynamically updating the interface based on user actions

The frontend was developed using **React 18** with **Vite** and **Tailwind CSS**, to create a fast and responsive interface.

Benefits:

- Component-based architecture for reusability
- Fast development and build process using Vite
- Efficient and consistent styling with Tailwind CSS
- Dynamic UI updates without full page reloads

We used **Zustand** for state management and **React Router v6** for navigation within the application. Additional libraries like **Axios**, **React Hot Toast**, and **Lucide Icons** enhance API communication, notifications, and UI design.

1.1.3 Controller: Handling Requests and Coordination

The Controller acts as an intermediary between the Model and the View. It processes user inputs, communicates with the Model, and updates the View accordingly.

Responsibilities of the Controller:

- Receiving and validating user input (e.g., login, signup, orders)
- Interacting with the Model to fetch or update data
- Sending processed data to the View
- Handling events such as form submissions and user actions

We used **Express.js**, a backend framework for Node.js, to manage routing and API development.

Advantages of Express.js:

- Lightweight and efficient
- Simple and intuitive routing system
- Supports middleware for request handling and validation
- Flexible for building RESTful APIs
- Easy integration with databases and third-party services

1.2 Integration of MVC with Layers

- **Model Layer**: Handles data operations, database access, and business rules
- **View Layer**: Responsible for presenting data to users
- **Controller Layer**: Manages communication between Model and View

1.3 Layered Business Logic

The Business Logic Layer handles core application functionality without directly interacting with the UI or database.

Key Components:

- **Application Services**: Manage high-level operations like order processing
- **Domain Services**: Handle specific logic such as recommendations or discounts
- **Validation Rules**: Ensure data correctness and enforce business rules
- **Data Integration**: Communicates with the database layer
- **Transactional Operations**: Ensures operations are completed successfully or rolled back

Flow:

1. Controller sends a request
2. Business Logic Layer processes it
3. Mongoose Model persists or retrieves the data

4. Controller updates the View

1.4 Development and Version Control Environment

We used **Git** as our version control system to track changes and manage collaboration. It allows multiple developers to work together efficiently and revert to previous versions when needed.
We used **GitHub** to host the repository and enable team collaboration, code sharing, and version management.

1.5 Authentication and Authorization

We implemented secure authentication using **JWT (JSON Web Tokens)** for session management.
User passwords are securely stored using **bcrypt**, which applies hashing and salting techniques to protect sensitive data.
We also used **Nodemailer** for sending OTPs and notifications, and integrated **Stripe** for secure online payment processing.

2 Codebase

An organization has been setup on GitHub for storing and collaborating on the source code of this project ***Qwik***
The repository contains the following files:

```
| README.md
|
+---qwik-server
|   | [.env.example, server.js, package.json]
|   |
|   +---src
|   |   | logo.png
|   |   |
|   |   +---config
|   |   |  [ constants.js, db.js]
|   |   |
|   |   +---controllers
|   |   |   [admin, auth, canteen, menu, order, payment, reservation, user controllers]
|   |   |
|   |   +---middleware
|   |   |   [auth, error handling, role middleware]
|   |   |
|   |   +---models
|   |   |   [Canteen, MenuItem, Order, Payment, Reservation, Review, User models]
|   |   |
|   |   +---routes
|   |   |   [admin, auth, canteen, menu, order, payment, reservation, user routes]
|   |   |
|   |   +---services
|   |   |   otp.service.js
|   |   \---utils
|   |       [apiResponse.js, asyncHandler.js]
|
\---qwik-client
    | [package.json, vite.config.js, index.html]
    |
    +---public
```

```
|   | logo.png
|
\---src
    | [App.jsx, main.jsx, index.css]
    |
    +---api
    |   [axios.js, index.js]
    |
    +---components
    |   | index.jsx
    |   |
    |   +---layout
    |   |   [AppLayout.jsx, AuthLayout.jsx]
    |   |
    |   \---sidebar
    |       [Admin, Customer, Merchant sidebars]
    |
    +---hooks
    |   [useAuth.js , usePolling.js]
    |
    +---pages
    |   | [authentication pages]
    |   |
    |   +---admin
    |   |   [dashboard, merchant management, approvals]
    |   |
    |   +---customer
    |   |   [home, hall selection, payment, reviews, party features]
    |   |
    |   \---merchant
    |       [dashboard, menu, orders, settings, party management]
    |
    +---store
    |   [authStore.js, cartStore.js]
    |
    \---utils
        constants.js
```

3 Completeness

3.1 Functionalities that have been implemented

The following functionalities specified in the Software Requirements Specification (SRS) have been implemented in the Qwik system. The system supports multiple user roles including Customer, Merchant, and Super Admin.

1. Customer Interface

The following customer-side functionalities have been implemented:

- **User Registration and Login:**
  Users can register and log in using their credentials.
  Authentication is handled securely using JWT-based tokens.
- **Browsing and Selection:**
  Users can browse available canteens and view food items along with details such as price.
- **Cart Management:**
  Customers can add items to the cart, update quantities, and remove items before checkout.
- **Order Placement:**
  Customers can place orders after selecting items. Orders are stored in the database and linked to user accounts.
- **Checkout and Payment Interface:**
  A payment interface is implemented (test mode) allowing users to simulate transactions.
- **Order History:**
  Users can view their past orders and track their activity.
- **Reviews and Feedback :**
  Users can provide feedback for orders.
- **Party Mode :**
  Users can place special orders(e.g., cakes) and can reserve tables for group dining.

2. Merchant Interface

The merchant-side functionalities include:

- **Merchant Registration and Login:**
  Merchants can create accounts and access their dashboard.
- **Order Management:**
  Merchants can view incoming orders placed by customers.
- **Accept/Reject Orders:**
  Merchants can accept or reject orders, which directly affects customer flow.
- **Order Status Updates:**
  Merchants can update the order status dynamically.
- **Menu Management:**
  Merchants can add, update, and manage food items available in their canteen.

3. Super Admin Interface

The administrator functionalities include:

- **Super Admin Registration using Invite Code**
  This is implemented to restrict unauthorized admin creation.
- **Super Admin Dashboard:**
  A centralized dashboard is implemented for monitoring system activity.
- **User Management:**
  Admin can view users and perform actions such as blocking or unblocking accounts.
- **Merchant Management:**
  Admin can monitor merchant accounts and control access.

4. Core System Features

- Communication between frontend and backend is implemented using REST APIs.
- Database operations (create, read, update, delete) are functional.
- Data is stored and managed using a database system.
- Environment configuration is handled using environment variables.

3.2 Non-Functional Requirements Achieved

Performance

- The system provides fast response times due to asynchronous request handling.
- Efficient API communication ensures smooth user experience.
- Capable of handling multiple users simultaneously during testing.

Security

- Passwords are securely hashed before storage.
- JWT-based authentication ensures secure access.
- Environment variables are used to protect sensitive data.

Usability

- The UI is simple and user-friendly.
- Navigation between pages is seamless using React Router.
- Feedback is provided using alerts and toast notifications.

Maintainability

- The project follows modular architecture (MVC + layered structure).
- Code is divided into frontend and backend for clarity.
- Components, controllers, and models are well-organized.

Scalability

- The system is designed to support future enhancements.
- Backend architecture supports addition of new APIs and features.

3.3 Future Development Plan

- The system currently uses a test/simulated payment interface. Future versions will include secure and real transaction processing
- Real time order status push using WebSockets instead of client side polling.
- SMS OTP for phone-number-based accounts.
- Refund flow for cancelled/rejected orders via Stripe Refunds API.

Appendix A - Group Log

Meeting notes and activities will be maintained here:

S.N	Team	From	To Date	Venue	Agenda
1	All	09/03/26	09/03/26	Hall12	Studied the template given and distributed the work amongst the team members. Also distributed the team according to the development roles.
2	Frontend Team	10/03/26	12/03/26	RM	Designed UI layouts and implemented pages using React (Login, Register, Home, Cart).
3	Backend Team	10/03/26	13/03/26	RM	Developed backend APIs for authentication, user management, and order handling using Node.js and Express.
4	All	13/03/26	13/03/26	Online	Integrated frontend with backend APIs and tested basic functionalities such as login and order placement.
5	Backend Team	14/03/26	17/03/26	RM	Implemented role-based access control, JWT authentication, and database integration.
6	Frontend Team	14/03/26	17/03/26	RM	Improved UI/UX, added cart functionality, and implemented dynamic rendering of data.
7	All	18/03/26	24/03/26	Online	Implemented merchant dashboard features, admin functionalities including blocking/unblocking users.
8	All	25/03/26	26/03/26	RM	System testing (customer → order → merchant accept → payment flow).
9	All	27/03/26	27/03/26	Online	Debugging issues, fixing bugs, and preparing final documentation.

Frontend Team: Prasun, Sumanth, Jashwanth, Yuvraj, Diya.
Backend Team : Chandrakanth, Ramalingam, Jashwanth Kumar, Sachin, Akhikesh.